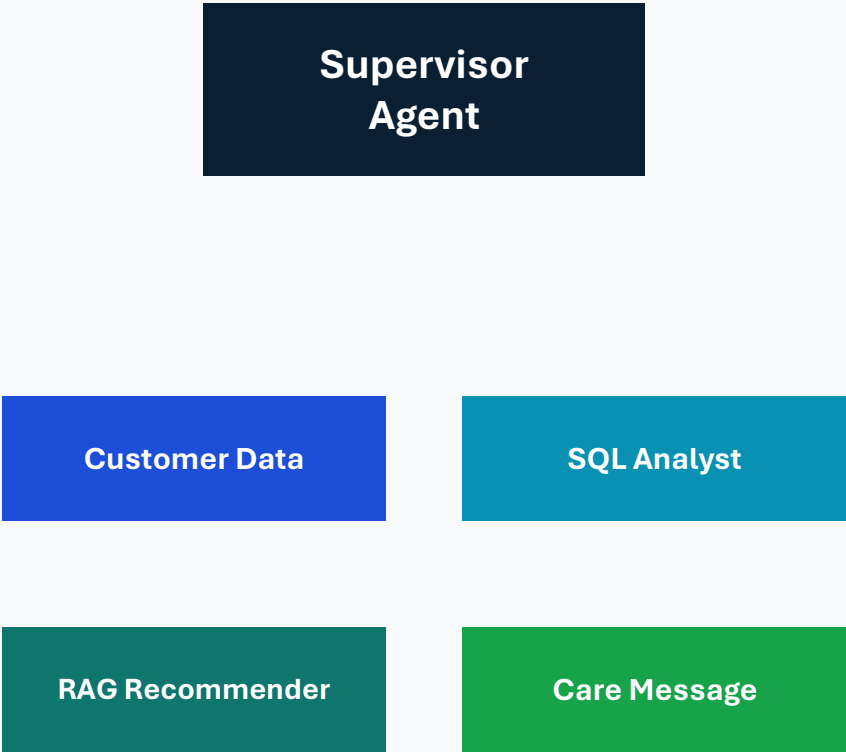
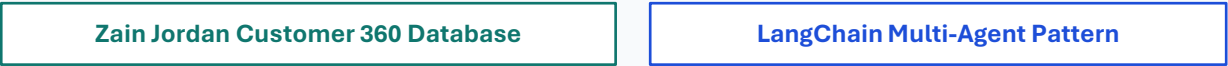


Multi-Agent Customer Care Copilot

Combining customer data, SQL analysis, RAG recommendations, and care messaging into one coordinated AI workflow.

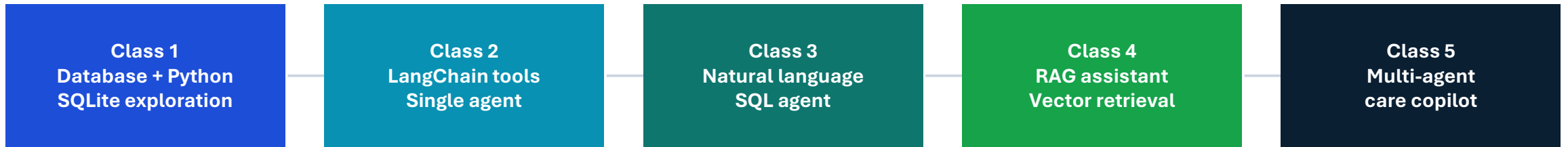


Formal training deck | No brand footer

Workshop Integration Class

Where Class 5 Fits

This is the integration class where prior components become one customer-care workflow.



Today's integration outcome

Participants see how database tools, SQL analytics, RAG recommendations, and message generation can work together through a supervisor agent.

Class 5 Goal

Build a practical multi-agent customer-care system around the Zain Jordan database.

What participants will understand

- Why one large agent can become difficult to manage
- How to split responsibilities across specialist agents
- How a supervisor coordinates subagents
- How prior classes connect into a real system

What participants will build

- Customer Data Agent
- SQL Analyst Agent
- RAG Recommendation Agent
- Care Message Agent
- Supervisor Agent that coordinates the full workflow

What this prepares them for

- Enterprise AI workflows
- Capstone architecture
- MCP tool exposure in the next class
- Business-ready AI demos

Why Move Beyond One Agent?

As workflows grow, one agent with many tools becomes harder to reason about and debug.

One-agent pressure

The same agent must retrieve customer details, perform SQL analysis, search recommendations, reason about churn, and write customer messages. This works for simple demos but becomes hard to inspect as complexity grows.

Multi-agent advantage

Each specialist agent owns a clear responsibility. The supervisor asks the right agent at the right time, then combines the evidence into a complete customer-care report.

Design principle: split by responsibility, not by hype.

One Agent vs Multi-Agent

Use one agent for simple workflows; use multi-agent when responsibilities become distinct.

One Agent

- Many tools in one place
- Fast to prototype
- Good for simple tasks
- Can become overloaded
- Harder to debug decisions
- Less modular for capstones

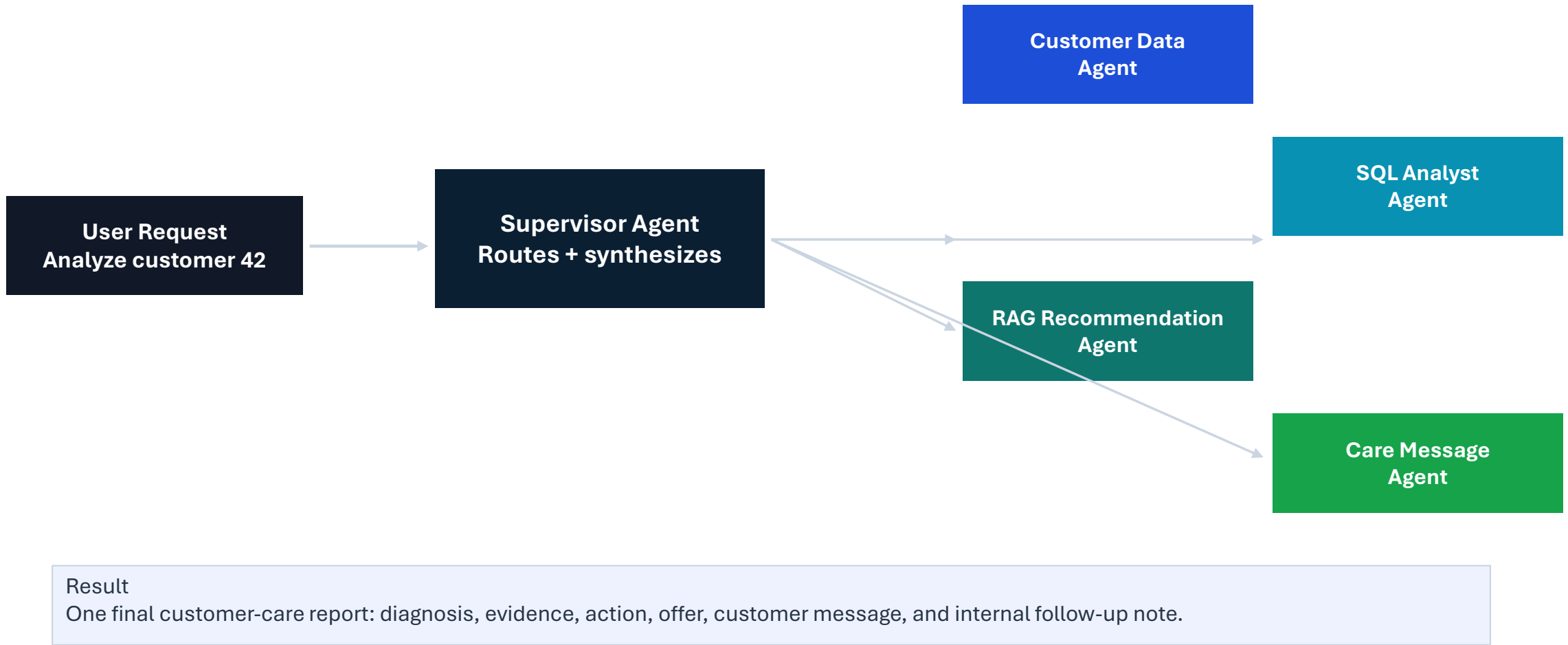


Multi-Agent

- Specialist agents
- Clear responsibilities
- Easier to inspect and test
- More reusable components
- Better for complex workflows
- Stronger capstone architecture

Multi-Agent Customer Care Architecture

A supervisor coordinates four specialist agents and produces the final report.



Specialist Agent Responsibilities

Each agent has a clearly defined job and output.

Customer Data Agent

Customer-specific facts: profile, plan, churn, value, complaints, support, billing, usage.

SQL Analyst Agent

Broad business context: churn by city, complaint patterns, revenue and campaign insights.

RAG Recommendation Agent

Semantic recommendations: plans, add-ons, campaigns, complaint themes, support feedback.

Care Message Agent

Communication: customer message, internal care note, professional call-script framing.

Supervisor Agent

Coordinates specialist agents, resolves the user request, and composes the final structured customer-care report.

Main Class Demo: Customer Retention and Care Copilot

The system analyzes a customer and turns evidence into a recommended action.

Demo prompt

Analyze customer 42. Check profile, plan, churn risk, value, billing, complaints, support history, usage, relevant offers, then draft a professional care message.

Inputs

Customer ID
Business question
Requested output sections

Agent work

Collect evidence
Analyze risk
Search recommendations

Output

Care report
Next best action
Customer message

Customer Data Agent

Retrieves customer-specific evidence from the Zain Jordan Customer 360 database.

Purpose

Get a complete customer 360 summary for a specific customer ID. It should not perform broad analytics or invent data.

Tools used

- get_customer_profile
- get_customer_plan
- get_customer_churn_risk
- get_customer_complaints
- get_customer_support_interactions
- get_customer_billing_summary
- get_customer_usage_summary
- get_customer_value_segment

Expected output

Customer summary, plan, churn risk, value segment, complaint history, support history, billing status, and usage pattern.

SQL Analyst Agent

Answers broader business questions using the natural language SQL agent pattern.

Best for

- Churn by city
- ARPU by segment
- Complaint volume
- Campaign conversion
- Network event impact
- Executive-style insights

Not for

- Writing customer messages
- Semantic recommendations
- Guessing reasons without data
- Unrestricted database changes

Guardrails

Only SELECT queries, schema inspection, row limits, business interpretation, no database modification.

RAG Recommendation Agent

Finds relevant context from plans, add-ons, campaigns, and customer-experience records.

RAG-friendly data

- plans
- addons
- campaigns
- complaints
- support_interactions
- customer_satisfaction

Best for

- Plan recommendations
- Add-on suggestions
- Campaign options
- Complaint themes
- Customer experience insights

Output

Structured recommendation with retrieved evidence and sources. If context is not enough, it says what is missing.

Care Message Agent

Transforms analysis into clear, professional communication.

Customer-facing message

Short, polite, professional, under 120 words, no internal churn-score language, no overpromised compensation.

Internal note

Brief follow-up note for the care team: issue, action, escalation need, and what should be checked next.

Why separate?

Communication quality matters. A dedicated message agent helps maintain tone, clarity, and policy-safe language.

Supervisor Agent

Coordinates specialist agents and produces the final customer-care report.

Responsibilities

- Understand the request
- Decide which agent to call
- Collect outputs
- Resolve evidence into a report
- Keep answer structured

Routing rules

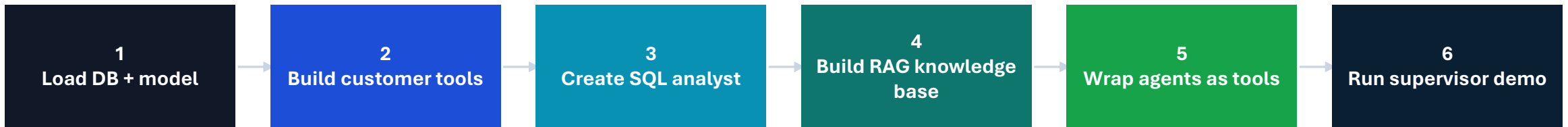
- Customer ID → Customer Data Agent
- Broad trends → SQL Analyst Agent
- Offers/recommendations → RAG Agent
- Message → Care Message Agent

Final format

Customer summary, issue diagnosis, churn/value risk, evidence, recommended action, suggested offer, customer message, internal notes.

How the Full Workflow Runs

The class notebook builds each component, tests it, then integrates under a supervisor.



Training principle

Build and test each specialist agent separately before connecting them. Multi-agent systems fail quietly when subagent roles are unclear or untested.

Notebook Structure

The class notebook mirrors the architecture in a beginner-friendly sequence.

Part A

Setup

API key, database, model,
helper functions

Part B

Specialists

Customer, SQL, RAG, Care
Message agents

Part C

Supervisor

Wrap subagents as tools and
coordinate final report

Result: a full multi-agent customer-care demo using the same Zain Jordan database.

Class Demo Flow

Four demos move from one customer analysis to comparison and business context.

Demo 1

Analyze customer 42 with all specialist agents and produce a full customer-care report.

Demo 2

Retention-focused analysis for customer 25: risk, complaints, offer, and care message.

Demo 3

Compare customers 25 and 42 to prioritize retention team action.

Demo 4

Add broader city-level churn trend using the SQL Analyst Agent.

Expected Final Customer-Care Report

The supervisor composes a structured answer that is useful for customer-care teams.

Participant Exercises

Exercises reinforce role selection, customer analysis, and capstone mapping.

Exercise 1: Right agent

Match user request to the best agent: customer facts, business trend, recommendation, or message writing.

Exercise 2: Customer analysis

Run the supervisor for customers 10, 25, 42, and 100. Compare urgency, risk, complaints, billing, and recommended actions.

Exercise 3: Capstone mapping

Choose a capstone idea and define required agents, tables, tools, final output, and demo script.

Capstone Connection

Class 5 provides a reusable architecture for team projects.

Churn Rescue Copilot
Prioritize high-risk, high-value customers and recommend retention action.

Billing Support Agent
Explain invoices, payment status, and next support action.

Campaign Recommendation Agent
Match segments to offers and draft outreach messages.

Network Impact Agent
Connect network events to affected customer-care priorities.

Executive Insight Assistant
Summarize churn, revenue, complaints, and campaigns.

Common Mistakes in Multi-Agent Systems

A strong architecture depends on clear roles and disciplined testing.

Design mistakes

- Too many agents too early
- Unclear agent responsibilities
- Supervisor doing all work
- Poor tool descriptions

Testing mistakes

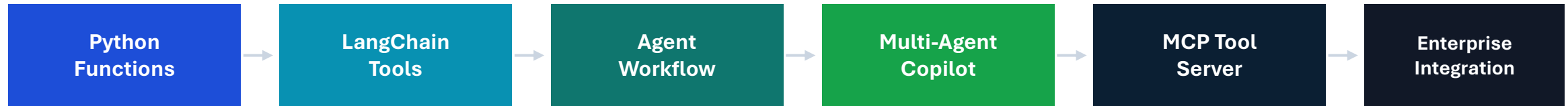
- Subagents not tested separately
- No inspection of intermediate outputs
- Hard-to-debug prompts
- No clear expected output

Data mistakes

- Using SQL for semantic questions
- Using RAG for calculations
- Guessing missing facts
- Ignoring row limits

What Comes Next: MCP

After building multi-agent workflows, the next step is enterprise tool integration.



Class 6 focus

Expose selected customer data and recommendation tools through MCP so agents and applications can access them through a standard protocol.

Class 5 Summary

A multi-agent system works best when each agent has a clear responsibility and the supervisor coordinates the final workflow.

Built today

- Customer Data Agent
- SQL Analyst Agent
- RAG Recommendation Agent
- Care Message Agent
- Supervisor Agent

Business value

- Faster diagnosis
- Evidence-based action
- Better customer messaging
- Reusable capstone architecture

Next step

Class 6 introduces MCP for standard enterprise tool integration.